Purser · Starter Kit 11 · x402 Business Agents

AI agents spend. You sign off.

AI agents can now spend company money; Purser lets them, through Airwallex and Metal, only within limits the CFO signs with Face ID.

The Kit 11 scenario, run live on 3 November.

24/24	**16 stops**	**5**	**6 s**
decisions by Claude, 0 fallbacks	nothing paid over a limit	decisions signed with Face ID (+1 co-signature)	from Face ID to PAID; the settlement is simulated in the sandbox

One live run, 3 November 2026: Metal testnet and the Airwallex sandbox, test money only.

Built on Airwallex and Metal, with Claude.

The problem

AI agents can now spend money. So who signs off?

- An agent pays as fast as it is told to, also by a fake invoice.
- A limit written in a prompt can be talked out of.
- **Purser: Inside its limits an agent pays alone; beyond them, the CFO signs.**

What only Purser has

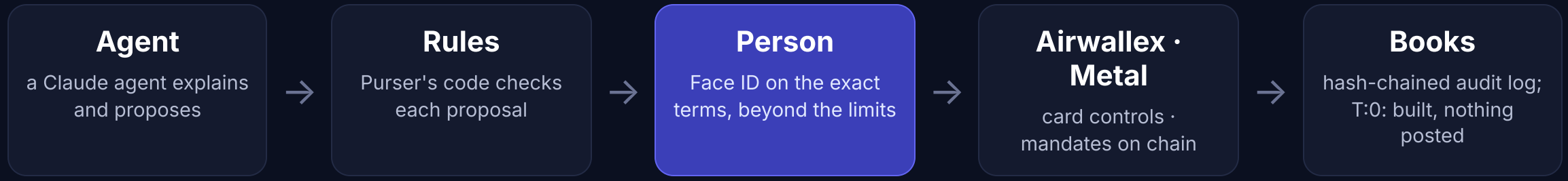
Each agent its own card and its own mandate

> An approval is usually a button. In Purser it's a signature over the exact terms: change one cent and it's worthless.

> One CFO policy becomes three locks: Purser's code, Airwallex's card controls, Metal's mandate on chain. A prompt is not a control.

> Real agents, not scripts: Claude made all 24 decisions, and no key reaches Claude.

Try it in 30 seconds. **kit11.max-31b.workers.dev/try/sign**



Northwind Outdoor, an outdoor wholesaler that sells live stock data to retailers per query. Three agents (Sourcing, Support, Treasury), 20.00 test USD between them.

The demo · try it yourself

Sign one yourself



kit11.max-31b.workers.dev/try/sign

A test world of your own. A rule blocks one payment; you sign the other with your passkey.

Change one cent, and the same signature is refused.

Test world · test passkey · nothing real moves.

The main run, 3 November 2026 · replay on /watch

One live run, three beats

1 · The price rise

Stockline 0.02 → 0.04, over the 0.03 cap

Metal refused it; the refusal is recorded on chain.

2 · The dispute

Only the 3 bad ones refunded

12 orders disputed; "refund the other 9" rejected by the CFO with Face ID.

3 · The euro bill

PAID 6 s after Face ID

EUR 110.00 bought by FX for only the shortfall; settlement simulated in the sandbox.

Treasury's options, within the same 20.00: A: Sourcing's cap 0.04, our price stays 0.05 · B: the same cap, our price 0.07. The CFO signed A.

Airwallex proof · sandbox, read back with GET only

The hash the CFO signed is in Airwallex's record

Airwallex (sandbox): the hash of the exact terms the CFO signed with Face ID is in Airwallex's own record of the EUR 240.00 SWIFT payment, and 3 of 3 agent cards carry the CFO's policy as Airwallex's own controls, read back from Airwallex.

The SWIFT payment (P-0005)

Signed with Face ID	`797d4483594f…dcababcd`
In Airwallex's record	`797d4483594f…dcababcd` **match**
Transfer	`03360bfa…3985`, SWIFT, EUR 240.00, PAID

Each agent's card: the CFO policy as Airwallex's own controls

- **Sourcing agent** ····1423: 5 of 5 controls match
- **Support agent** ····6203: 5 of 5 controls match
- **Treasury agent** ····6279: 5 of 5 controls match

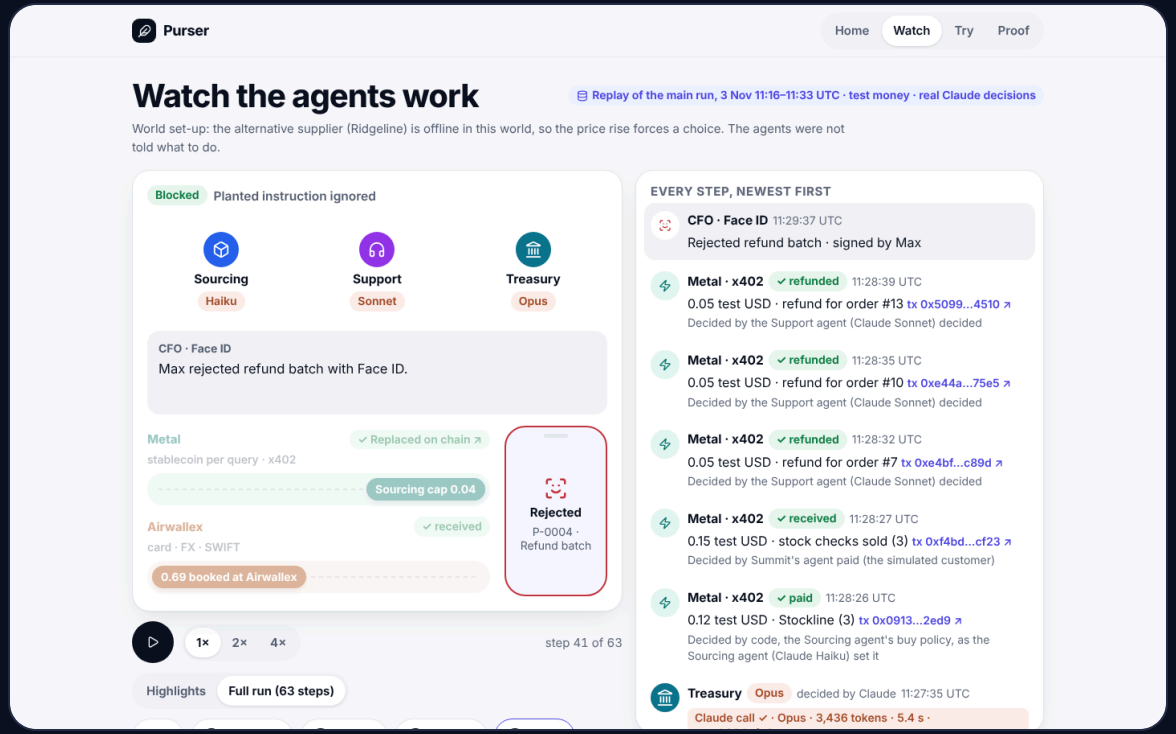
Limit per purchase and per month, merchant category, currency, end date.

Read on 2026-11-06 from the sandbox: `docs/runs/airwallex-transfer-approval-2026-11-06.json` and the cards' record beside it. Also on /proof and /airwallex.

Metal proof · testnet

Every id is published

- **Metal (testnet): a mandate on chain per agent; 86 payments, each its own transaction; purchase #24 refused on chain (mandate_cap_exceeded); the mandates replaced on chain after the CFO's Face ID (four eyes; second approver a labelled test passkey).**
- 16 stops, nothing paid over a limit: a buy rule Claude set stopped 12 overpriced checks, Metal refused 2 purchases, the CFO rejected 1 proposal, and 1 planted instruction changed nothing. Metal's 2: 1 over the mandate's cap, 1 on a mandate that had just been replaced.
- On chain, checkable: Metal's refusal `0x2e92f9cb…1871`; the mandate replacements `0x789abb24…ebbc`, `0x2a91ac97…e790`; the sweep `0x86936a5b…ccb4`
- Published: FX `3ca88196…4636`, SWIFT `03360bfa…3985`, deposit `b3812e05…e0a0`; card raise P-0006, EUR 30.00 → 46.00, then Airwallex authorization `01a1a12f…0007`

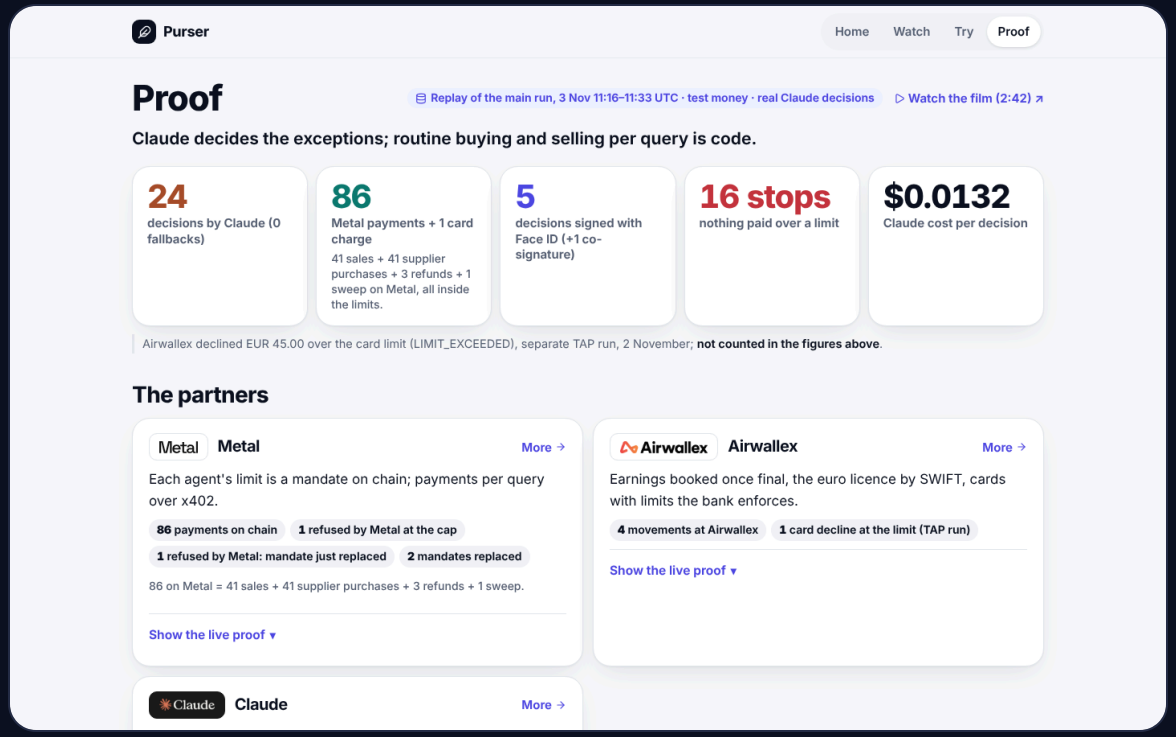


The main run, 3 November (/watch): #7, #10, #13 refunded, each on chain; "refund the other 9" rejected by Max with Face ID

The agents · Claude, measured

Claude explains and proposes

- **Claude: 24 of 24 decisions in the main run, 0 fallbacks; a reliability run (rel-1) found a fault in how the agents hand off a card raise, and after the fix the eval rebuilt from that moment went from 6/10 to 10/10 and the next run (rel-1b) passed every step.**
- 50 calls, USD 0.3170 at list prices. Sourcing on Haiku, Support on Sonnet, Treasury on Opus.
- Our benchmark: Haiku alone is as good on everyday cases, and misread one harder dispute.
- No key reaches Claude, and Claude approves nothing; code checks each proposal first.



/proof, the main run: 24/24 decisions by Claude; 86 Metal payments + 1 card charge; $0.0132 = Claude's cost per decision

Security and tests

Checked in code, on every push

2,634
tests in 267 files, all passing (npx vitest run)

412
audit rows in the main run, hash-chained: npm run verify

37/39
red-team attacks stopped, on stand-ins

18/18
Scenario Lab stories as expected, on stand-ins

- Red team on stand-ins: 37 of 39 attacks stopped, 2 more detected afterwards, not prevented.
- No key reaches Claude: tested, no key, token or credential ever reaches a model.
- A signature fits one set of terms: the server refuses it once one cent, the IBAN or the currency changes.
- CI on every push: typecheck, lint, the suite, the browser checks in Chromium and WebKit, CodeQL, Semgrep, and gitleaks over the whole history.

Said plainly

What is real, and what is not

Real, with test money

- The Airwallex sandbox: cards, FX, SWIFT and the deposit.
- Metal testnet: 86 payments and 6 mandate transactions, on chain, checkable; no clearing in this run.
- Claude: 24 of 24 decisions, each with its request id.
- Max's passkey (Face ID on his phone); Max plays the CFO.

Ours, simulated or not yet live

- The supplier, the customer and the TAP merchant are run by us.
- SWIFT SENT and PAID are moved by Purser through the sandbox's simulation API.
- The second approver was a labelled test passkey, pressed by Claude Code on Max's go.
- Built, not yet live: the Cardputer hub. T:0 bookkeeping is built; posting failed, so nothing is posted.
- The reliability runs were signed by labelled virtual test passkeys, no person.

The business · a hypothesis

Why this is a company

Not validated with customers yet: no interviews, no pilots, no revenue. Every line below is a hypothesis except the cited source.

- **The problem:** when an AI agent pays today, its limit lives in a prompt it can be talked out of, or in a month-end review that finds the money after it has left.
- **Who buys:** the finance team of a small company or scale-up that already banks with Airwallex and lets AI agents pay; the CFO uses it and decides.
- **Intended pricing:** €49 per company a month including 5 agents; €10 per extra agent (hypothesis).
- **Why now:** Airwallex wrote on 5 October 2026 that agents moving money need limits enforced "through underlying infrastructure rather than prompts" ("Agentic banking starts with governance", source S1).
- **Why Purser:** it builds on Airwallex's own card controls and Metal's mandates, and adds what sits on top: one CFO policy for card and chain, approvals signed over the exact terms, a hash-chained audit log.

Sources and reasoning: docs/business/ in the repository.

Roadmap · a plan, not a promise

From test rails to production

Now

Airwallex sandbox · Metal testnet · test money · one company per ledger

Next

test the demand with finance teams · production Airwallex · a real second approver · keys in a KMS, Postgres, a pager on /health

Then

Metal mainnet · four eyes by default · an outside SOC 2 audit

Claude explains and proposes, the rules in code hold every limit, and a person signs before money moves beyond them.

Try it	kit11.max-31b.workers.dev/try/sign
Every id	kit11.max-31b.workers.dev/proof
Code	github.com/agamemnon-coder/x402-business-agents
Offline, no keys	`npm ci && npm run judge`

Visa TAP: built and verified against Visa's public reference verifier; not eligible Visa tooling for this hackathon (Visa, 5 Nov), so Purser is entered for the Metal Award only.